

RBT AI 训练营 · 第 2 周课件（第一次课）

这是给学生分享用的合并版，由 `scripts/build_full_courseware.sh` 从拆分课件生成。

维护时请修改原始拆分文件：`training-notes/curriculum/week-02-01.md`。

PRD + GitHub Issues + 造一个 skill 服务自己的工作流

第一次课 · 约 2h（时间以群通知为准）

spec-driven 起步 + skill 创建 demo + Q&A

课前必读（学员请在第一次课上课前看完）

W1 复盘：你本来应该做完什么

打开你 W1 的 PR，检查：

- 私有 repo + 至少 1 个 AI 协助 commit
- `CLAUDE.md` v0 / `AGENTS.md` v0 / `README.md` v0
- 工具箱 6 个 `--version` 截图
- 本地能跑的 hello-world
- `docs/reports/W1-<your-id>.md`

任何一项缺失请在第一次课前补齐。W2 所有内容都假设你已有能跑的项目和基础 `CLAUDE.md`。

这周你要做什么

W1 你证明了“能跑通 AI 工具”。W2 你要证明“能让 AI 跑通工程流程”。

1. 把模糊的项目想法变成一份结构化 **PRD**
2. 基于 PRD 出一版低保真原型 + 讨论校准（写代码前先把需求误会修掉）
3. 把 PRD 拆成 ≥ 8 个 **GitHub Issue**（每个有验收标准）
4. 为自己造一个能跑通的 **custom skill**（建议 `/draft-issue` / `/check-prd` / `/seed-data` 任选）

完成这几件事，你就有了 "AI 能持续帮你推进"的项目骨架，后面 8 周都建立在这上面。

这周不做什么

- 不做 业务代码 (W3 才开始)
- 不追求 skill 复杂——30 行 markdown 跑通就行
- 不追求 PRD 漂亮——v0 能让 AI 拆 issue 就行，每周迭代

W2 高风险提醒

W2 是整门课内容超载风险最高的一周。怎么处理：

- 第一次课听明白机制就行，不必当堂学会写 PRD
- 第二次课 lab 追求"跑通 + 提交"，不追求"写得好"
- 方法论清单只是点名，后续每周深入，不要试图本周掌握全部
- skill 跑通即可，复杂模式留给 W6
- 项目可以很小，甚至只是简历展示页或登录页；但必须跑完整流程

第一部分：W1 复盘 + W2 课程总览

1.1 W1 在 W2 起什么作用

W1 本质是给 AI 准备工作环境。W2 起这些"环境"开始变成 AI 的输入：

W1 产出	W2 怎么用
CLAUDE.md v0	写 PRD 时 AI 知道项目是什么
AGENTS.md v0	Codex 读到同样上下文
README.md v0	"3 个核心功能"是 PRD 起点
hello-world commit	工具链通畅，本周专注流程
W1 report	写 W2 report 时有基线

Insight：W1 的 CLAUDE.md / README.md 现在要升级成更结构化的输入——这就是 PRD。

1.2 W2 在 10 周里的位置

W1 立项 → **W2 PRD + 原型 + Issues + skill** → W3 数据库 + 后端 + TDD → W4 前端（原型细化） → W5 部署。

W2 是"项目从想法到 backlog"的转折点。W3 起所有代码工作都从 W2 拆出来的 issue 开始。所以 W2 拆得越规范，W3-W5 越省力。

1.3 最小开发工作流长什么样

W2 不是比谁功能多，而是比谁真正把流程摸通。最小也可以是一个登录页：

1. 写清楚登录页需求
2. 让 AI 生成页面或代码
3. 本地跑起来，或打开静态 HTML 预览
4. 自己 review：是不是符合需求，有没有明显问题
5. `git add` / `git commit` / `git push`
6. 开 PR 或合并到主分支

这条链路每一步都可能坑：需求不清、AI 做偏、本地跑不起来、Git 不会提交、PR 不会合并、权限没配好。先用小功能把坑踩完，比一开始做大系统更重要。

1.4 本周关键词

- **spec-driven**：先写结构化 spec（PRD / issue / acceptance criteria），再让 AI 写代码
- **acceptance criteria（验收标准）**：每个 issue 必须有"做到什么算完成"
- **skill**：你自己定义的、Claude Code 里 `/<name>` 调用的工作流模板
- **front-matter**：skill 顶端 YAML 元数据，告诉 Claude Code 这 skill 何时触发

第二部分：PRD 是什么 + 为什么必须写

2.1 PRD 不是"产品经理的活"

工程师常把 PRD 当"产品经理的废话文档"。这是 pre-AI 时代的偏见。

AI 时代 PRD 的角色变了：它是给 AI 交底的——AI 按 PRD 拆 issue / 写代码 / 写测试 / 写 review。PRD 是项目最重要的上下文文件之一。

简单说：**PRD 是项目的"长上下文版 CLAUDE.md"**。CLAUDE.md 是 1 屏速览，PRD 是 3-5 屏细节。

2.2 PRD 必须包含的 5 个板块

板块	写什么
目标	解决什么问题、给谁解决
非目标	明确不做什么（防 scope creep 最有效手段）
用户故事	"作为 X，我想 Y，以便 Z"
验收标准	做到什么样算完成（给 AI 和未来自己一把尺）
风险	已知会卡的 + 不确定的

用一个具体例子展开。

真实项目里的 PRD 可能更复杂：review 状态、MVP 范围、领域模型、权限、数据模型、API、UI、验收标准都可能单独成段。**W2 先守住这 5 个板块**，不要一上来追求完整产品团队文档。

2.3 一个 PRD 反例（不要这样写）

```
1 # 我的项目 PRD
2 ## 目标：做一个好用的工具，帮助大家更方便地解决问题。
3 ## 功能：用户注册登录 / 各种功能 / 数据展示
4 ## 技术栈：React + Python + 数据库
```

问题："好用"/"更方便"AI 读完毫无信息；"各种功能"没法拆 issue；没有用户/场景/验收/非目标。喂给 Claude 拆 issue，要么得 8 个空话 issue，要么 AI 反问你 20 个问题。

2.4 一个 PRD 正例（好 PRD 长这样）

```
1 # Roommate Finder for UNSW International Students – PRD v0.1
2
3 ## 1. 目标
4 帮 UNSW 国际新生（2026 T1 入学的 Chinese / Indonesian / Indian 学生）
5 在到校前 2–6 周找到合规、可信、价格透明的合租室友。
6 衡量指标（v1.0 上线 4 周内）：≥ 50 注册 / ≥ 10 对成功配对 / 反馈分 ≥ 4.0/5.0。
7
```

```
8  ## 2. 非目标
9  - 不做房源中介（不接长租）
10 - 不做通用社交（无 timeline / 无帖子）
11 - v1.0 不做移动 app（只有 web）
12 - 不做支付 / 押金托管
13 - 不接收非 UNSW 学生注册（邮箱白名单）
14
15 ## 3. 用户故事
16 - 作为刚拿到 offer 的国际新生，我想填偏好问卷并看到匹配排序，以便落地前约 2-3 人视
    频聊
17 - 作为已在悉尼的在读生，我想发布"找室友"信号并 30 天自动过期，以便不被陈年帖子骚扰
18 - 作为查看匹配的用户，我想看到"作息一致、预算重叠 80%、都不抽烟"的可解释打分，以便
    决定要不要发起对话
19
20 ## 4. 验收标准 (v1.0)
21 - 邮箱白名单（仅 unsw.edu.au / ad.unsw.edu.au）
22 - 问卷至少 10 项；匹配算法必须输出可解释 reasoning（不是黑箱）
23 - dashboard 显示"我发出/收到的对话"
24 - 部署在 Cloudflare，公网 URL 可访问，首屏 LCP < 2.5s
25
26 ## 5. 风险与未知
27 - UNSW 是否允许第三方用 unsw.edu.au 白名单？需查 IT policy
28 - 匹配算法 v1.0 用规则 + 余弦；反馈差是否升级到 LLM-based？
29 - profile 公开度（默认匿名？）—— 待定
30 - 撮合行为是否需要法律免责声明？
```

为什么这个好：目标用户精确；衡量指标可验证；非目标砍掉了 4 个 scope creep；用户故事可拆 issue；验收标准 review agent 能自动检查；风险板块诚实。

这就是你 W2 第二次课要写的东西。

2.5 PRD 不是写一次就完了

PRD 是活文档。W2 写 v0.1；每周 review，发现理解错用户或要砍功能时，**改 PRD + commit + 写进 report.md**。不要悄悄改代码不改 PRD——这是 slop-driven 的起点。W10 Demo Day PRD 会被读出来作为评委入口。

两个管理习惯从 W2 就开始：

- 以后再做的东西不要散落在 PRD 正文里。统一放到 `docs/backlog.md`、`docs/TODO.md` 或 GitHub Issues；否则几周后你会忘，AI 也会误以为它们是当前版本要做的。
- 同一个功能只保留一个真相来源。不要到处新建 PRD 副本。如果必须拆文档，要在主

PRD 里明确链接和版本关系；否则 PRD v0.1 写一套、PRD v0.2 写另一套，AI 会同时读到冲突上下文。

2.6 PRD 定稿后，先出原型、再拆 issue（本周固定环节）

PRD 把基本需求用文字定下来了——但**文字不是画面**。很多需求误会，读 PRD 看不出来，一看到具体页面立刻发现"这不是我要的"。所以 W2 的工作流是固定的三步：

1 | PRD（文字定基本需求） → 原型（画面校准） → 拆 issue（进开发）

为什么 PRD 之后、拆 issue 之前要先插一道原型：

- **PRD 是文字，原型是画面。** 在写一行代码之前就把"文字里看不出的误会"暴露并修掉，省下大量返工。
- **原型是讨论的载体。** 拿一个能点的原型找讲师/同学讨论，比对着一段 PRD 文字高效得多。
- **低保真就够。** W2 只要"低保真原型"——能看清页面结构和主要交互即可，不追求像素级精美（高保真细化留到 W4 前端开发前）。
- **原型反过来改 PRD。** 如果原型暴露了 PRD 的问题，回头改 PRD——这正是**先出原型**的意义。
- **反馈要具体。** 不要只说"不好看"；要说"登录按钮放到表单底部"、"红色太强，降低饱和度"、"侧边栏保留 5 个菜单项"。

怎么出原型（AI 原型工具，迭代很快、以当前可用为准）：

工具	一句话
手绘 / 截图 / 白板图	最低成本，只要能看清页面结构和主要交互就可以
Claude 的设计功能 (Claude Design)	Claude 桌面端/网页版自带、已和 Claude 额度打通，能出页面/原型，传统后台界面也能做得不难看
OpenAI Sites (@sites)	OpenAI 新功能，用自然语言直接生成网站/落地页/原型
Open Design	开源版 Claude Design，能力定位类似、可自部署，不想被某家额度绑住时用
让 AI 直接 mock HTML	最轻量、不依赖任何平台，让 Claude/Codex 直接吐 2-3 个 HTML 原型挑一个

这一节怎么落地（第二次课 lab 会做一遍）：

- 基于 PRD 用户故事，让 AI 出一版低保真原型（主要页面 + 核心交互）
- 存进 `docs/prototype/`（截图 / 链接 / mock HTML 任一形式）
- 和讲师/同学讨论一轮：哪里和你脑子里的不一样？需求要不要改？
- 把"讨论后改了什么"记到 `report.md` 或 `docs/decisions/`；暴露了 PRD 问题就回头改 PRD
- 校准完，再进入第四部分拆 issue

这道"原型 → 讨论"会在 **W2 末尾**正式做一遍（PRD 收口后），并在 **W4 前端开发前**再细化成高保真一版。详见 W1 课件「心法 #5」。

第三部分：spec-kit + 用 AI 辅助写 PRD

3.1 spec-kit 是什么

[spec-kit](#) 是 spec-driven 开发模板和工具，提供 spec / plan / tasks 三层结构 + 引导 AI 的 prompt 模板 + 与 GitHub Issues / Claude / Codex 的集成。

W2 用它的精神，不强制用它的 CLI：

- 用：spec → plan → tasks 三层心智模型；它的 PRD prompt 思路
- 不强制：装 spec-kit CLI、照抄它的目录结构

W2 lab 只要求手写一份 **PRD + 拆 issue**，路径轻量。

3.2 用 AI 写 PRD 的 prompt 模板

不要直接说"帮我写一份 PRD"。AI 会编。**正确做法是迭代式 prompt：**

第一步：让 AI 问你问题、不是写答案

- 1 我要给 [UNSW 国际新生] 做 [找室友 web 工具]。我现在脑子里只有这一句。
- 2 请扮演资深 PM，向我提 10 个问题来澄清：
- 3 真实用户细分 / 核心场景 / 必做 vs 不做 / 衡量指标 / 已知风险
- 4 每个问题加 1 句"为什么问这个"。不要现在就写 PRD。

AI 问、你答。一轮做完，AI 对项目的理解就和你对齐了。

第二步：起草 PRD 骨架

- 1 基于上面回答，生成 PRD v0.1 (5 板块：目标/非目标/用户故事/验收标准/风险)。
- 2 目标必须有衡量指标；非目标 ≥ 4 项
- 3 用户故事 ≥ 3 个 (作为 x 我想 y 以便 z)
- 4 验收标准必须可被自动 review 验证 (具体数字/文件/URL)
- 5 风险板块含"已知会卡"和"不确定"两类
- 6 不要"未来可能"、"也许"这种模糊语言。

第三步：让 Codex 挑刺

- 1 codex
- 2 > 扮演挑剔的早期投资人，读这份 PRD，找 5 个最严重问题：
- 3 > 哪里不够具体 / 哪些"目标"其实是"功能" / 哪些验收标准无法验证
- 4 > 用户故事有什么 implicit 假设 / 风险板块漏了什么

带反馈回去改 PRD。

Prompt 反模式："帮我写一份 PRD" (太空) / "写一份好用的 PRD" (不可测) / "参考某大厂模板" (cargo cult) / "把想法变 PRD" 不给细节 (AI 只能编)。

3.3 superpowers 的 /brainstorming 和 /writing-plans

装好 superpowers 后：

- /brainstorming —— 模糊想法 → 结构化 (项目刚开始用)
- /writing-plans —— PRD → 可执行多步骤 plan (拆 issue 前用)

不要"每个项目先跑 5 个 skill"——那是 cargo cult。

第四部分：从 PRD 到 GitHub Issues 的拆解规范

W2 推荐用 GitHub Issues，而不是只在本地文档里列任务。原因很简单：Issue 可以被 AI 读取、引用、拉下来实现，也天然接上 branch / PR / review / merge 的开发流程。

4.1 一个 issue 必须包含什么

title (一句话) / body (背景 + 用户故事 + 任务清单 + 验收标准) / labels (类型 + 优先级 + 模块) / milestone (如 v1.0) / assignee (通常自己)。

4.2 label 规范 (W2 就立起来)

- 类型: `feat` / `fix` / `chore` / `docs` / `refactor` / `test`
- 优先级: `p0` / `p1` / `p2`
- 模块: `backend` / `frontend` / `infra` / `ux`
- 状态 (可选): `blocked` / `needs-design` / `ready`

规则: 每个 issue ≥ 1 个类型 + 1 个模块 + 1 个优先级 label。

4.3 issue body 的标准模板

```
1  ## 背景
2  [为什么要做, 引用 PRD 哪一节]
3
4  ## 用户故事
5  作为 [X], 我想 [Y], 以便 [Z]。
6
7  ## 任务清单
8  - [ ] 设计 x 的 schema
9  - [ ] 写 endpoint POST /api/x
10 - [ ] 写测试覆盖正常 + 1 个边界
11
12 ## 验收标准
13 - POST `/api/x` 接受 `{ "name": "..."}` 返回 `201` + 新对象 id
14 - 空 name 返回 `400` + error message
15 - 测试覆盖率  $\geq 70\%$ ; CI 绿
16
17 ## 相关
18 - PRD §3 用户故事 #1; 依赖 #3
```

4.4 issue 反例

`title: 做用户登录` / `body: 实现一下登录功能` / `labels: 无`

问题: 没背景没验收; AI 不知道做 OAuth / Email / SSO; 没有"完成"标志; 没 label 无法被 milestone filter。

4.5 PRD $\rightarrow$ issue 拆解的 prompt 模板

```
1 读这份 PRD。拆成 ≥8 个 GitHub Issues:
2 1. 每个 issue 服务于某用户故事或验收标准
3 2. 可独立完成（不依赖未拆的工作）
4 3. 含背景 / 用户故事 / 任务清单 / 验收标准
5 4. label: 类型 + 模块 + 优先级
6 5. 验收标准必须可被自动 review 验证（数字 / 文件 / URL / 响应码）
7 6. milestone: v1.0
8 7. 不要重复 PRD 内容; issue 是 PRD 的"可执行子集"
9
10 输出: 每个 issue 一段 markdown, --- 分隔。
11 最后给表格: 序号 / title / labels / 预估工时。
```

跑完后**必须人工 review**: 验收标准真的可验证? 任务清单有没有漏? 真的可独立? 通过后用 `gh issue create` 批量创建或在 web UI 开。

4.6 8 个 issue 是怎么来的

W2 要求 ≥8 个不是魔法数字: < 5 拆得太粗; 5-7 偏粗; **8-15 sweet spot**, 每个 1-4 小时能做完; > 20 太细缺用户视角。

W2 还没动业务代码, 8 个 issue 大概长这样 (找室友项目):

#	title	label
1	feat(backend): user schema + email whitelist	backend/feat/p0
2	feat(backend): POST + GET /api/users/me	backend/feat/p0
3	feat(backend): preference questionnaire schema	backend/feat/p0
4	feat(backend): matching algorithm v1 (规则+余弦)	backend/feat/p0
5	feat(frontend): 首屏 + 邮箱注册 UI	frontend/feat/p0
6	feat(frontend): 偏好问卷页	frontend/feat/p0
7	feat(frontend): 匹配列表 + profile 详情	frontend/feat/p1
8	infra: CI + Cloudflare 部署流水线	infra/chore/p0
9	docs: API contract (OpenAPI v0)	docs/chore/p1

注意: 没有"做用户登录"这种空洞 issue——每条都对应 PRD 的具体功能或验收标准。

4.7 大功能先当 epic，再拆 issue

如果一个需求太大，例如"用户登录系统"，先把它当成一个 **epic**，再拆成多个可执行 issue：

- 前端登录页
- 后端登录 API
- 用户表 / migration
- 表单校验与错误提示
- 登录测试
- 文档与运行说明

不要把这些全部塞进一个 issue。一个 issue 应该小到可以 review：做了什么、怎么测、是否完成，都能被清楚判断。

第五部分：自定义 skill 创建

5.1 skill 是什么

skill 是放在 `.claude/skills/<name>/SKILL.md` 的 markdown 文件。Claude Code 在某些场景自动调用，或你用 `/<name>` 显式调用。可理解为"可复用 prompt + 行为说明 + 可选脚本"。

最实用的创建方式：先完整跑通一次会话，再让 AI 把这次成功流程总结成 `SKILL.md`。也就是说，skill 不是一开始凭空写出来的，而是把一个已经跑通的好 prompt / 好流程沉淀下来，下次不用重新解释。

和现有 skill 集的关系：

来源	是什么
内置	Claude Code 自带（如 <code>/init</code> ）
superpowers	通用 skill 集（ <code>/brainstorming</code> / <code>/writing-plans</code> / <code>/test-driven-development</code> ）
gstack	web 开发 + 部署 skill 集（ <code>/browse</code> / <code>/qa</code> / <code>/ship</code> ）
你自己的	服务于你这个项目特有工作流的 skill

核心区别：superpowers / gstack 给所有人用；你的 skill 给你这个项目用。

5.2 skill 文件结构

```
1 | your-project/.claude/skills/draft-issue/SKILL.md
```

也可以扁平 `.claude/skills/draft-issue.md`。

5.3 SKILL.md 的格式

```
1 | ---
2 | name: draft-issue
3 | description: 把一段 PRD 片段或粗略需求描述，转成符合本项目规范的 GitHub Issue
  | 草稿。
4 | trigger: when the user wants to draft a new GitHub issue from PRD text
  | or a feature idea
5 | ---
6 |
7 | # /draft-issue
8 |
9 | ## 这个 skill 做什么
10 | 读用户提供的 PRD 片段或需求描述，输出符合本项目规范的 issue 草稿。
11 |
12 | ## 输入
13 | - 一段 PRD 文本，或一句话需求描述
14 |
15 | ## 输出格式
16 | ---
17 | ## 背景
18 | [引用 PRD 哪一节]
19 |
20 | ## 用户故事
21 | 作为 [X]，我想 [Y]，以便 [Z]。
22 |
23 | ## 任务清单
24 | - [ ] ...
25 |
26 | ## 验收标准
27 | - ...
28 |
29 | labels: feat / backend / p1
```

```
30 milestone: v1.0
31
32
33 ## 必须遵守
34 - 验收标准必须可被自动 review 验证（数字/文件/URL/响应码）
35 - label 必须含：类型 + 模块 + 优先级
36 - 不能编造 PRD 里没有的需求
37 - 信息不足时反问用户，不要编
38
39 ## 反模式
40 - 输出"实现一下登录功能"这种空洞描述
41 - 把多个用户故事塞到一个 issue
42 - 用"让体验更好"、"提升性能"这种模糊语言
```

5.4 front-matter 字段说明

字段	作用	必须
name	skill 的名字，会被识别为 <code><name></code>	✓
description	一句话说明这个 skill 干什么	✓
trigger	什么时候 Claude 应该考虑自动调用这个 skill	推荐

`trigger` 字段很关键。写得好，Claude 会在合适的时机主动建议用这个 skill；写得差，它就只会在你显式 `/draft-issue` 时才用。

5.5 settings.json 配置基础

`.claude/settings.json` 是项目级配置文件。W2 只关心一件事：告诉 Claude Code 去哪里找 skill（一般默认就够，不写也行）。

```
1 { "skills": { "paths": [".claude/skills"] } }
```

更复杂的 `permissions` / `hooks` / `mcp` W6 进阶再讲。W2 不需要。

5.6 三个推荐 skill（任选一个写）

skill	输入	输出	谁适合
<code>/draft-issue</code>	PRD 片段 / 需求描述	一份合规的 issue 草稿	大多数人
<code>/check-prd</code>	一份 PRD 文件	一份 review 报告（缺哪些板块、哪些验收标准不可测）	想把 review 自动化的人
<code>/seed-data</code>	schema 描述 / table 名	seed 脚本（生成合理的 dummy data）	后端偏好型

W2 建议选 `/draft-issue` ——因为你正好要拆 8 个 issue，这个 skill 写完立刻派上用场。

5.7 测试 skill 能跑

```
1 cd your-project && claude
2 > /draft-issue
3 > 用 PRD §3 用户故事 #1 起草一份 issue
```

Claude 说"找不到 skill"时：检查路径 → front-matter `name` 与 `/<name>` 一致 → 重启 Claude。

5.8 一个 skill 反例

```
1 ---
2 name: helper
3 description: 帮我做事
4 ---
5 让 Claude 帮我做项目相关的事情。
```

问题：name 太空、description 不可触发、body 没说输入输出。**skill 不能是"通用 helper"**，必须有明确的输入 → 输出 → 反模式。

第六部分：开发方法论速览（只点名，不展开）

W2 第一次课花 10 分钟把这些"方法名"过一遍。目的不是掌握全部，而是以后听到能对号入座。后续每周需要时会展开。

6.1 课程主线深入覆盖（必须会）

方法	何时深入	一句话
Spec-Driven	本周起步、W8 大 feature	先写结构化 spec，再让 AI 拆 issue / 写代码
TDD	W3	red-green-refactor；AI 时代变种：让 AI 先写测试
GitHub Issues-Driven	本周起步、W9 完整规范	issue → branch → PR → review → merge

6.2 节奏型方法

- **Plan-Driven**：大 feature；brainstorm → plan → 人工 review → AI 执行（`/writing-plans`）
- **Walking Skeleton**：早期；先跑通端到端最小链路（W1 hello-world 就是）
- **Pair Programming with AI**：探索阶段；不适合严密设计的功能
- **Vibe Coding**：周末 / 一次性；进生产前必须升级到 spec-driven

6.3 从产物倒推（W4 再点名）

- **Documentation-Driven**：先写文档再写实现
- **Schema-First / Contract-First**：先定 OpenAPI / GraphQL schema（W4 在用）
- **Type-Driven**：TS / Rust；先定类型实现填空
- **Demo-Driven**：从最终 demo 倒推 backlog（W10 暗含）

6.4 协作型方法（W7 深入）

- **Multi-Agent Orchestration**：不同 AI 扮演不同角色
- **Continuous AI Review**：每个 PR 跑 AI review（W5 起就做）
- **Trunk-Based**：短 branch、频繁 merge 到 main
- **PR-Driven**：每个变更是小 PR；PR description 是变更 spec

6.5 反模式（必须避开）

W9 会深入批判，W2 先立名字：

反模式	症状	何时勉强 OK
Slop-Driven	不 review 直接接受 AI 输出 → 下次看不懂自己代码	一次性脚本
Vibe Coding	早期爽、后期改不动	周末实验；生产绝不
Cargo Cult Prompting	抄网上"魔法 prompt"不理解	永远不
YOLO Mode	不测试、不 staging、直接推生产	永远不
AI Review Bypass	选择性忽略 Critical	永远不；可申诉，不能装看不见

6.6 选择口诀（决策时翻）

- 明确需求 + 团队 + 生产 → spec-driven + TDD + issue-driven（课程主线）
- 复杂大 feature → spec-driven 之上嵌套 plan-driven
- 不熟悉的领域 → 先 walking skeleton 跑通，再迭代
- 纯 prototype → vibe coding OK，别推生产
- API / 前后端协作 → schema-first
- 开源项目 → documentation-driven
- 任何时候 → 不 slop、不 YOLO、不 cargo cult

第七部分：第一次课 Live Demo（讲师演示）

讲师将现场完整跑一遍 "PRD → 拆 issue → 造 skill" 这条链路。请认真看，第二次课 lab 你要重复一遍。

7.1 Demo 脚本（讲师在大屏 type 这些）

```
1 # 0. 假设在已有 hello-world 的 demo 项目里
2 cd ~/projects/roommate-finder
3
4 # 1. 启 Claude, 让它问出 PRD 的关键问题
5 claude
6 > 我要给 UNSW 国际新生做找室友的 web 工具。我现在脑子里只有这一句话。
```

```
7 > 请扮演资深 PM, 向我提 10 个问题澄清:
8 > 真实用户细分 / 核心场景 / 必做 vs 不做 / 衡量指标 / 已知风险
9 > 每个问题加 1 句话"为什么问这个"。不要现在就写 PRD。
10
11 # [讲师当场回答 10 题, 让学员看清"PRD 是被问出来的"]
12
13 # 2. 起草 PRD
14 > 基于上面回答, 生成 PRD v0.1 (5 板块), 写到 docs/PRD.md:
15 > 目标必须有衡量指标 / 非目标 ≥ 4 项 / 用户故事 ≥ 3 个 (作为 x 我想 y 以便 z)
16 > 验收标准必须可自动验证 / 不要"未来可能"、"也许"
17
18 # 3. Codex 挑刺
19 codex
20 > 扮演挑剔投资人, 读 docs/PRD.md 找 5 个最严重问题。
21
22 # 3.5 出低保真原型 + 现场讨论 (PRD 定稿后、拆 issue 前)
23 claude --continue
24 > 基于 docs/PRD.md 的用户故事, 出一版低保真原型 (主要页面 + 核心交互),
25 > 存到 docs/prototype/。我要拿去讨论。
26 # [讲师现场拿原型讨论: 哪里和脑子里的不一样? 需求要不要改?]
27 # 暴露了 PRD 问题就回头改 PRD——让学员看清"先出原型是为了在写代码前修掉误会"]
28
29 # 4. 校准完, 把反馈带回去让 Claude 改 PRD, 然后拆 issue
30 claude --continue
31 > 读 docs/PRD.md, 拆 ≥8 个 GitHub Issues:
32 > 每个 issue 服务于某用户故事或验收标准
33 > 含背景 / 用户故事 / 任务清单 / 验收标准
34 > label: 类型 + 模块 + 优先级
35 > 验收标准可自动验证 / milestone: v1.0
36 > 每个 issue 用 --- 分隔。最后给表格: 序号 / title / labels / 工时。
37
38 # [讲师当场 gh issue create 创建几个]
39
40 # 5. 当场造 /draft-issue skill
41 mkdir -p .claude/skills/draft-issue
42 claude --continue
43 > 在 .claude/skills/draft-issue/SKILL.md 创建 skill:
44 > 作用: 输入 PRD 片段, 输出符合本项目规范的 issue 草稿
45 > front-matter: name / description / trigger
46 > body: 这个 skill 做什么 / 输入 / 输出格式 / 必须遵守 / 反模式
47 > 参考刚刚拆 issue 的模板做输出格式。
48
```

```
49 # 6. 测试
50 claude
51 > /draft-issue
52 > 用 PRD §3 用户故事 #2 起草一份 issue
53
54 # 7. commit + milestone
55 git add docs/PRD.md docs/prototype/ .claude/skills/draft-issue/
56 git commit -m "feat(spec): add PRD v0.1, low-fi prototype and /draft-
issue skill"
57 git push
58 gh api repos/:owner/:repo/milestones -f title="v1.0" -f
description="First public release"
```

7.2 重点观察

- AI 是怎么"问出" PRD 的（不是 AI 写、是你被引导想清楚）
- Codex 挑刺抓到了 Claude 的什么盲区
- 原型一出来，文字 PRD 里看不出的需求误会怎么立刻浮现（这是先出原型的价值）
- 拆 issue 时 prompt 的约束和输出格式起到什么作用
- skill = 把好 prompt 固化——文件就是这么提炼出来的

7.3 讲师会现场强调的 3 件事

1. PRD 不是写出来的，是被问出来的——没有 §3.2 第一步那 10 个问题，PRD 就是空话
2. skill 不是"通用 helper"——必须有明确输入 / 输出 / 反模式
3. W2 跑通这条链路就够——后面 8 周不断加菜

第二次课 Lab（独立文档）

本周第二次课的动手任务、当周产出与验收标准见独立文档：

👉 [week-02-02.md](#)

第十部分：课后自学清单（两次课之间）

必读：

1. spec-kit 仓库 README (看 spec / plan / tasks 三层结构)
2. 一份优秀开源项目 PRD / RFC (Notion / Stripe API RFC、Linear "How we build")
3. superpowers `writing-skills` 文档 (如果装了)
4. GitHub Docs: Issues / Labels / Milestones 前 3 节

浏览即可：

5. Anthropic 官方 Claude Code skill 文档
6. 一份 superpowers 内置 SKILL.md (看 front-matter / body 写法)
7. `~/.claude/skills/gstack/skills/` 任选 1-2 个 SKILL.md 对照

鼓励试一下：

8. 把 PRD 给同学/讲师看 5 分钟，看他能不能复述项目目标
9. 用你的 skill 提前拆 W3 issue (体会 skill 复利)
10. 用 `/check-prd` (如果你写了) 跑一次自我 review

下一次课开始前必做：写 `docs/reports/W2-<your-id>.md`。

附录 A：W2 命令速查

```
1 # Claude / Codex
2 claude / claude --continue / codex
3
4 # GitHub milestone + label + issue
5 gh api repos/:owner/:repo/milestones -f title="v1.0" -f
  description="..."
6 gh label create feat --color 0e8a16
7 gh issue create --title "feat(backend): user schema" \
8   --milestone "v1.0" --label "feat,backend,p0" --body-file
  /tmp/issue1.md
9 gh issue list --milestone "v1.0"
10
11 # skill
12 mkdir -p .claude/skills/draft-issue
13 $EDITOR .claude/skills/draft-issue/SKILL.md
14 claude
15 > /draft-issue
```

```
16 > [PRD 片段]
17
18 # 验证
19 gh issue list --milestone "v1.0" | wc -l # 应 ≥ 8
20 ls docs/PRD.md .claude/skills/draft-issue/SKILL.md
21
22 # commit
23 git add docs/ .claude/ CLAUDE.md
24 git commit -m "feat(spec): add PRD v0.1, 8 issues, /draft-issue skill"
25 git push
```

附录 B：常见问题 FAQ

Q: 我项目还没想清楚就写 PRD 是不是太早？

A: 正因为没想清楚才需要写。用 §3.2 第一步的 10 个问题逼自己想。PRD v0.1 不是终稿，每周可改。

Q: PRD 写中文还是英文？

A: 都行。挑团队 / AI 都看得懂的一种。不要中英混杂（AI 会困惑）。

Q: 不用 spec-kit 自己手写行吗？

A: 行，且推荐。spec-kit 是参考。W2 只要求 `docs/PRD.md` 满足 §9 验收标准。

Q: 8 个 issue 是硬性最低吗？我项目小，5 个够了。

A: 8 个硬性最低。"项目小"通常意味拆得不够细——比如"前端 + 后端 + 部署"塞一个 issue。拆开。

Q: skill 要做复杂吗？

A: 不要。30 行 markdown 就够。复杂结构留给 W6。

Q: skill 创建后 Claude 没识别？

A: 检查路径 → 重启 Claude Code → 确认 front-matter `name:` 和 `/<name>` 一致 → 检查 `.claude/settings.json` 没排除 skill 目录。

Q: 用 superpowers 的 `/brainstorming` 写 PRD 行吗？

A: 行。但自己再过一遍别 slop-driven。commit 前你要能复述每一段。

Q: PRD 写完发现方向不对要重来吗？

A: 微调 → 改 PRD + commit；大改方向 → 立刻找讲师 1:1。越早改越省。

Q: 能把多个用户故事塞一个 issue 吗?

A: 不能。一个 issue 一个用户故事是底线。

Q: skill 文件能从别的项目搬吗?

A: superpowers / gstack 等通用 skill 可以装，但项目特有 skill（/draft-issue）必须自己写——因为它包含你项目的 label / PRD 路径 / issue 模板。

Q: 能让 AI 帮我写 SKILL.md 吗?

A: 当然。但 SKILL.md 是高密度 prompt 工程产物——你必须读懂每一段，否则跑不准时不知道改哪。

附录 C：Troubleshooting

现象	解决
<code>/draft-issue</code> 找不到 skill	重启 Claude；检查路径；确认 front-matter <code>name: draft-issue</code>
skill 跑通但输出格式不对	SKILL.md 里给完整输出模板代码块，别只写"按规范输出"
<code>gh issue create</code> 权限错	<code>gh auth refresh -s repo</code>
Claude 写 PRD 全是空话	回 §3.2 第一步，让 AI 先问 10 个问题再写
Codex review 全是 nitpick	prompt 里加角色（"挑剔投资人"）+ 5 个明确问题
多个用户故事被塞一个 issue	prompt 加"一个 issue 一个用户故事"
issue body 没验收标准	prompt 加"必须含验收标准，且可被自动 review 验证"
PRD 写了 2 小时还没拆 issue	强制跳下一步；PRD 是活文档
skill 堆 200 行 AI 还不按规范	砍冗余；核心约束放最前；加反模式对照
<code>.claude/settings.json</code> 导致启动失败	删掉用默认； <code>jq . settings.json</code> 检查语法

更多直接问讲师/助教。

第一次课下课前 Check-in

请每人在群里回复：

1. 你打算写 PRD 的项目用户群体是？一句话描述。
2. 你打算造 `/draft-issue` / `/check-prd` / `/seed-data` 哪一个 skill？为什么？
3. 你 W1 的 6 项产出（repo / CLAUDE.md / AGENTS.md / README.md / hello-world / report）补齐了吗？
4. 你最担心 W2 的哪一步？

讲师会在第一次课晚之前回复所有担心点。

第二次课下课前 Check-in

请每人在群里回复：

1. 你的 `docs/PRD.md` URL（指向 repo 里的文件）
 2. milestone `v1.0` 下 issue 数量（用 `gh issue list --milestone "v1.0" | wc -l` 取数）
 3. 你造的 skill 名字 + 是否真的跑通了（是 / 否 / 卡在哪）
 4. 一句话 retrospective："这一周让 AI 帮我拆 issue 最有用 / 最坑的瞬间是 ____"
-
-

下周预告

W3 我们正式进入数据库 schema 设计 + 后端骨架 + TDD。

具体来说：

- 让 AI 画 ER 图、写 migration（spec-driven 的 schema 段）
- 跑通 TDD red-green-refactor 节奏：让 AI 先写测试再写实现
- 跑通 GitHub Actions CI，每次 push 自动跑测试
- 引入"backend agent"概念——给 AI 一个角色化上下文，让它专注后端

W3 起，你 W2 拆的那 ≥ 8 个 issue 就是 backlog。你每完成一个 issue $\rightarrow$ close 一个 $\rightarrow$ 离 v1.0 milestone 近一步。

别忘了：下一次课开始前提交 `docs/reports/w2-<your-id>.md`。这是从 W1 起的硬性产出，AI 协作质量 15% 主要靠它评。
